Lecture 1: Introduction

– prerequisite know *C*

```c
#include <stdio.h>

int main() {
  printf("Hello, World!");
  return 0;
}
```

Assessment

Assignment	25%
Assignment	25%
Exam	50%

Labs

– Lab 1: Compulsory, Introduction to Assignment
– Lab 2: ???

Embedded/cyber-physical systems (ES/CPS)

Embedded System

– employs a combination hardware & software to perform a specific function
– is part of a larger system that may not be a 'computer'
– works in a reactive and time constrained environment
– is not meant to be programmed by a user

CPS to emphasise interaction with the physical world.

ES became key in mechanical and mechatronic devices in 1990s-2000s
– IoT, IIoT

Uses software for flexibility. Hardware for performance & security.

Have both hardware and software components:
– Hardware: microprocessors, ASIC, VLSI, FPGA, communication, signal processors
– Software: Embedded operating system or real-time operating system and bare metal

Need to satisfy *functional* and *non-functional* – low power, security, reliability, safety – requirements.

Common Characteristics

Application specific	made for a specific application
Use digital signal processing	process signals in digital form
Reactive	react on events from physical world
Real time	perform functions within time constraints

Communicative	communicate with other systems
Distributed	part of spatially distributed system
Human Computer Interface	communicate with humans

Embedded System Applications

- Wireless systems
- Data communication systems
- Vehicle automation
- Intelligent sensors
- Home and building networking and automationo
- Control and instrumentation
- Entertainment

FPGAs good for making custom computing machines – processors, accelerators.

difference between validation and verification.

Types of embedded systems

- Reactive or control dominated systems – Data and events arrive in irregular intervals and must be processed within strict timing constraints
- Data dominated – data arrives in regular streams

Dependability

Reliability $R(t)$	probability of system working correctly when it was working at $t = 0$
Maintainability $M(d)$	probability of system working corrrectly time d after an error occurs at $d = 0$
Availability $A(t)$	probability of system working at t
Safety	no harm caused to people or resources
Security	confidential and authentic communcation

Efficiency

- Energy
- Code-size
- Run-time
- Weight
- Cost

“A real time constraint is called hard, if not meeting that constraint could result in a catastrophe”

Lecture 2: From Specification to Design

System-Level Tasks

Implementation
- Task concurrency management: what tasks need to run at the same time
- High-level transformations: find candidates for hardware implementation by looking at initial program on microprocessor
- Design space exploration: explore possible solutions and weigh options
- Hardware/Software partitions: interfacing between hardware and software & what components are reponsible for what behaviours
- Compilation
- Scheduling

Hardware vs Software

Hardware Functionality implemented using customized architecture
Software Functionality implemented on a programmable processor

Key Differences
- processors usually have a single thread of control
- hardware may have multiple parallel processes

Design

Specification capture
- The description of the model in a language

Explore Deisgn Alternatives
- Allocation of system components
- Specification of their physical and performance constraints

Specification refinement
- Allocations of variables to memories
- Insertion of interface protocols between components
- Addition of arbiters to resolve conflicts from access common resources by concurrent processes
- Specification of processors

Models of Computation

Concurrent models
- Kahn process networks
- Logic gates
- Communicating sequential processes
- Synchronous-reactive
- Globally asynchronous locally synchronous

Sequential models
- FSM
  - Mealy: Output is function of input and state
  - Moore: Output is function of state
- HFSM

– Register machines

Activity Orentied Models
– Data flow graphs
– Control flow graphs

ES Design & Specification and Programming Languages
– VHDL & Verilog for Hardware description
– Programming languages (sequential and concurrent) used for software functionality

Validation
– Completeness: includes all possible input sequences the environment to the system
– Correctness: generates expected output for every such input sequence

Internet enabled Frequency Relay
– Measures current frequency and its rate of change in power signal
– Disconnects loads if frequency is not nominal (like 50 Hz)
– Communicates with other supervisory and control systems through landline or wireless system
– Operates non stop
– Low cost for use at household level

Power dissipation in CMOS Gates

$$P_{sw} = \frac{C_{\text{load}} V_{\text{dd}}^2}{T}$$

$$I_{\text{leak}} = I_{\text{sub}} + I_{\text{ox}}$$

$$I_{\text{sub}} = K_1 W e^{-\frac{V_{\text{th}}}{n V_\theta}} \left(1 - e^{\frac{V_{\text{th}}}{V_\theta}}\right)$$

$$I_{\text{ox}} = K_2 W \left(\frac{V}{T_{\text{ox}}}\right)^2 e^{-\alpha \frac{T_{\text{ox}}}{V}}$$

Reducing static power consumption
Logic Design
– Multiple V_{th}
– Multiple T_{ox}
– stack effect:

Runtime
– Using sleep transistors
– Variable V_{th}
– Multiple V_{dd}
– Application sensitive input control

Reducing dynamic power consumption
Logic Design
– Proper transistor sizing
– Multiple V_{dd}
– Optimising logic design

Runtime

- Clock gating: disable or reduce clock signal for certain component or whole system
- Dynamic voltage scaling
- Dynamic frequency scaling
- Dynamic thermal management

Design issues for MPSoC

- High level of performance (processing time)
- Power management and optimisation (reduce heat)
- Reducing the energy consumption (for battery life)
- Fault tolerance (for safety critical applications)
- High level of system security (in particular for IoT)

Lecture 2

Platform Designer

Platform Designer is a system integration tool. It integrates different IP cores into the FPGA and automatically makes interconnection logic.

Interconnection

Usually designed on point to point interconnect for efficient communication. Signals:

- data
- addr
- ready
- rd (read)
- valid
- ready
- ack (acknowledged)

Lecture 3

Structural Models: show intersections of the system

Requirements of Specification Languages

Hierarchy of functional modules	simplifies top down decomposition and bottom up integration can be structural or behavioural (functions, modules in software)
Concurrency	describes concurrent operations and parallelisms
State transitions	model internal systems state; important for control systems
Exceptions	interaction with environment – current state terminated and transition into another state
Communication and synchronization	exchange of data between concurrent parts – shared memory, message passing
Program instructions	data types, decision constructs, assignments
Time	describe timing constraints – reponse time

RTOS

- simulate concurrency on single processor
- RTOS manages multiple running processes
- processes perform the system's computation and the RTOS schedules them
- attempts to meet deadlines by deciding which process runs when

RTOS Scheduling

- most RTOSs use fied priority preemptive scheduling where each process is given a particular priority when the system is designed
- RTOS runs the highest priority process that is expected to stop after a short period of time
- priorities are usually assigned using rate monotonic analysis which assigns higher priorities to processes that must meet more frequent deadlines

SystemJ

`emit S`: emit signal S
`present S`: if signal S is present do

- presence of signal is high if it was emitted in the last tick
- value of signal works how you expect me
- If different value of the same signal is emitted in the same tick the last emission is taken to be the value for the signal for the next tick

Channels

- channels must only have one sender and reciever
- channels are named
- `send channel_name`, `recieve channel_name`

– channels are unbuffered

Kernel Statements
– `pause` marks the end of the tick
– `input` or `output` parameter for signals is from the environment
– `[input|output] [type] signal identifier`
– loops in systemJ must take one tick
– `abort` is preemption (basically interrupts) based on signal
– `suspend` stop execution while signal is present
– `trap(identifier) p; do q` basically software interrupts if `exit(identifier)` is executed in `p` do `q`
– `p || q`: run `p` and `q` synchronously in parallel, waits for threads to finish
– `#identifier` extracts value from signal or channel

```SystemJ
output char signal thing;
emit thing[23];
```

`CD_name (input|output) -> p`: declares clock domain with name `CD_name` with specified I/O ports and starts `p` as the root reaction

clock domains have their own scopes

Lecture 3: RTOSs

RTOSs

Operating system that simply handles multiprocessing and multithreading ('Task').
Implements

- Scheduler
- Inter-task sync and comms

System tick that maintains time-based services like task execution interval and time slicing using a hardware timer.

Why use?

- Abstraction of hardware
- Multiprocessing & Multithreading
- Resource management
- Guarantees meeting real time constraints
- Services
  - Timer
  - ISR
- Common
- Application organization and modularity
- Common communication stacks and drivers

Basics

RTOS Kernel schedules tasks

Tasks have their own state

- Task has atomic completion and non atomic completion (atomic being whether the execution can be separated)
- Priority of tasks is given by designer

Tasks

Task = Code + Data + State (context)

Task state is stored within a Task Control Block that contains the

- ID
- Priority
- Status
- Registers
- Saved Program Counter
- Saved Stack Pointer

State Diagram for Task

TODO: steal diagram of slide

Implementations of RTOS

Monolithic Kernel

- Statically linked when compiling
- Monolithic binary
- Compile time optimizations & analysis possible

– Hard to change, must change entire binary

Micro Kernel
– It's all system calls!
– Limited things handled by kernel other things done in userspace
– Easy to change as a result
– Less optimizations

RTOS Standards

TODO: I was too slow

– RT-POSIX: POSIX OS with real time extensions
– OSEK: automotive

RTOSs examples

Tiny
– TinyOS
– Contiki
– NuttOS

Medium
– microCOS-II/III
– eCos
– FreeRTOS
– ThreadX
– microLinux

Large
– VxWorks
– QNX Neutrino
– Real time linuxes
– Research kernels: SHARK, MARTE

RTOS

– Implemented in ANSI C (damn so it works on like everything with a cc)
– open source code
– <10 kB
– Priority can change during runtime

Has:
– Tasks
– Queues
– ISR communication
– Semaphores and mutexes

FreeRTOS Features

– Implemented in C
– Fixed priority, pre emptive scheduling
– Traps software interrupts
– Traps timer interrupts

API functions for:
- Creations and management of multiple tasks
- Inter task communication through queues, semaphores and mutexes
- Heap memory mangement through malloc and free

ISRs are independent from tasks

Communication
- Queues (channels)
- Binary and counting semaphores
- Mutexes

Lecture 3: FreeRTOS OS

Data Types & Coding Style

- camel case used for unsigned
- screaming snake case used for signed
- only use `long` and `short`
- use hungarian notation

Start variables and functions with prefix for:

`char`	`c`
`short`	`s`
`long`	`l`
`portBASE_TYPE`	`x`
`unsigned`	`u`
`pointer`	`p`
`void`	`v`

- second word in name is source file
- `vTaskPrioritySet()` returns `void` and is defined in `task.c`.

Task Priorities

`configMAX_PRIORITIES - 1`

Task Creation

```c
portBASE_TYPE xTaskCreate(
  pdTASK_CODE pvTaskCode,
  const signed char* const pcName
  unsigned short usStackDepth,
  void* pvParamters,
  unsigned portBASE_TYPE uxPriority,
  xTaskHandle* pxCreatedTask
);
```

> Whoever the hell decided `for(;;)` should exist should be shot. I'm gonna use it anyway though.

- get and set priorities while task is running
- suspend and resume tasks
- suspend all tasks useful for interrupts
- task delay
- task delete

Kernel Structure

`taskENTER_CRITICAL()` and `taskEXIT_CRITICAL()` provide a basic critical section implementation that disables intrrupts up to a priority

- `xQueuePeek` reads the top of the queue but does not remove it
- there are special functions for using queues from an ISR

Binary Semaphores
- binary = boolean
- `give` sets the semaphore to 1
- `take` sets the semaphore to 0

Counting Semaphores
- queue multiple events, do thing x number of times